#hackthekitty 2026

Project Report

Project Name: Cat Adoption Profile Optimiser

Reference ID: UWAAV5WM

---

1. Executive Summary

Cat Profile Optimiser is a machine-learning web app that helps shelters and owners write better cat adoption listings so cats get adopted faster. A user enters a cat's details and instantly receives a 0-100 adoption-success score, a SHAP explanation of what is helping or hurting that score, actionable recommendations, an AI-rewritten description, and a live "what-if" re-score that shows the score climb as the listing improves. By turning an opaque prediction into concrete, listing-level advice, the tool helps reduce time-to-adoption and free up scarce shelter capacity.

2. Project Overview

2a. Why you're building what you're building

Cats with poorly constructed adoption listings (missing details, weak or very short descriptions, too few photos, unconfirmed vaccination or sterilisation status) tend to wait longer to be adopted. Longer waits tie up limited shelter space and staff time. Shelter workers are stretched thin and don't always know which parts of a listing matter most. This project addresses that gap: instead of guessing, a user gets a data-grounded score plus specific, actionable steps to improve the listing.

2b. How it relates to the theme

The hackathon theme is HackTheKitty. The project is a cat-focused tool end to end: it is trained on real cat adoption data (the PetFinder.my dataset, filtered to cats and single-cat listings) and its entire purpose is to help individual cats get adopted faster. The theme is integral to the problem, the data, and the solution, not a surface-level mention.

2c. Target Audience

The primary users are animal-shelter staff and coordinators who create adoption listings at volume and want to improve them quickly. A secondary audience is individual rehomers (for example, someone rehoming a litter) who have never written an adoption listing and want guidance on what to include and how to phrase it.

3. Key Features

- ☐ Adoption score (0-100) from an XGBoost model, shown as a clear headline metric.
- ☐ Per-listing explanations, which include a SHAP-based diverging bar chart showing what is helping vs hurting the score, in intuitive score-points.
- ☐ Actionable recommendations (targeted, threshold-aware advice, only for factors a shelter can actually change).
- ☐ AI-improved description, which is an LLM rewrite of the listing text that uses only the facts provided (no invented claims).
- ☐ “What-if” re-score, including interactive sliders that re-score the listing live and show a before/after delta as the listing is improved.

4. Technology Stack

Layer	Technology
Language	Python 3.11
ML Model	XGBoost (native categorical support)
ML Tooling	scikit-learn (splitting, metrics, calibration checks)
Explainability	SHAP (TreeExplainer)
Experiment Tracking	MLflow (training only)
Front-End / UI	Streamlit
Charts	Plotly (diverging SHAP bar chart)
LLM Rewrite	Google Gemini API (gemini-2.5-flash, via Google AI Studio) (powers AI recommendations and description rewrite)
Data	PetFinder.my Adoption Prediction (Kaggle)
Serialisation	joblib (model + JSON schema)
Version Control	Git/Github
Security Scanning	Aikido
Testing	pytest (15 unit tests)
Deployment	Streamlit Community Cloud

5. Technical Architecture

The system is organised into a clean, layered pipeline. The ML engine is separated from the UI and from the "intelligence layer," so each part is independently testable and replaceable.

Data/request flow: the user enters a cat's details in the Streamlit sidebar. On submit, app.py assembles the inputs into a single cat record and passes it through preprocess.py, which produces model-ready features. model.py loads the pre-trained XGBoost model and returns a 0-100 score (predicted probability x 100). explain.py wraps a SHAP TreeExplainer to turn that prediction into a ranked list of contributing factors, which drives both the on-screen explanation chart and the recommendations. The app calls Google's Gemini API for two AI features: personalised recommendations (grounded by the SHAP factors) and a description rewrite. The what-if feature re-runs the same preprocess -> score pipeline on a modified copy of the cat. The what-if feature re-runs the same preprocess -> score pipeline on a modified copy of the cat to show a live before/after.

Key design decision: a single preprocess() function is shared by both training and inference, so the features produced at prediction time exactly match those used at training time (no train/inference skew). The trained model and a JSON feature schema are saved together, so the app loads a self-contained artifact and never trains at runtime.

Component summary:

- ☐ app.py (Streamlit UI and orchestration; loads the model, schema, and explainer once at startup. The personalised recommendations and the description rewrite are in this file.)
- ☐ src/preprocess.py (shared feature engineering (train + inference), feature schema, category handling.)
- ☐ src/model.py (training, 0-100 scoring, and model persistence (joblib).)
- ☐ src/explain.py (SHAP wrapper: per-cat explanation and global feature importance.)

(A visual architecture diagram is included in the README.)

6. Testing Matrix

The project has 15 automated unit tests (pytest) across the three engine modules, plus manual UI checks. Automated tests cover the most critical properties, including the train/inference consistency guarantee and a regression test for the security fix, and behaviour is verified with sensible sanity checks (e.g. more photos should not lower the score).

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Train/Inference Consistency	Preprocess the same cat as a dict and as a DataFrame; compare output columns/shape	Identical FEATURE_ORDER columns and shape (no skew)	Identical	Pass
Derived Features	Preprocess a cat with a known description and Fee=0	desc_word_count and is_free computed correctly	Correct	Pass
Empty Description	Preprocess a cat with an empty description	0 words, no error	0 words	Pass
Categorical dtype	Preprocess a cat; inspect Breed1/Color1 dtype	category dtype (required by XGBoost)	category	Pass
Missing Feature	Preprocess a cat with a required field removed	Fails loudly (raises)	Raised	Pass
Path-Traversal Guard	Call save_schema with a path outside the project root	Rejected with an error (security fix)	Rejected	Pass
Score Range	Score a valid cat	Score within 0-100	In range	Pass
Score Determinism	Score the same cat twice	Identical score both times	Identical	Pass
Behavioural Sanity	Score a cat with 0 vs 4 photos	More photos does not lower the score	Held	Pass
Model Round-Trip	Save then reload the model; score a cat	Score identical after reload	Identical	Pass
Explanation Contract	Explain a cat; inspect factor dicts	Each factor has feature/label/value/impact/direction; sorted by |impact|	Correct	Pass
Global Importance	Compute global SHAP importance	All features returned, sorted by importance	Correct	Pass
App (Score Flow) (manual)	Enter a cat in the sidebar; click Analyse	Score + interpretation shown	Works	Pass
App (Explanation) (manual)	After analysing, view the SHAP chart	Diverging bar chart of helping/hurting factors	Works	Pass
App (What-If) (manual)	Drag the photo/description sliders	Score re-computes live with a before/after delta	Works	Pass
App (Graceful Fallback) (manual)	Run app before partner modules exist	Recommendation/LLM sections show placeholders, app doesn't crash	Works	Pass

7. Security

A dedicated SECURITY.md documents responsible data handling (public dataset, no user input persisted), secrets management (the single Gemini API key is kept out of the repo, read from an environment variable, verified with git ls-files), secure coding (input validation via the form, graceful failure of external calls), and dependency hygiene (a lean, pinned requirements list). An Aikido scan found no critical or high issues and no secrets; the one project-code finding (a theoretical path-traversal in save_schema) was fixed with a verified path-validation guard, and the remaining findings were triaged as dependency advisories or out-of-scope account-surface items.

8. Future Improvements

- ☐ **Photo-Quality Analysis:** Let users upload a cat photo and receive quality feedback (brightness, sharpness, framing), since photo count is one of the strongest levers in the data.
- ☐ **Ordinal/Multiclass Modelling:** Predict the full adoption-speed bucket (with Quadratic Weighted Kappa) alongside the binary score, to preserve more nuance.
- ☐ **Region-Specific Models:** The current model is trained on one dataset (primarily Malaysia). Retraining or fine-tuning per region would improve generalisation.
- ☐ **A/B-Style Listing Comparison:** Score two versions of a listing side by side to make the improvement explicit.

9. Tools You Used

- ☐ **AI Coding Assistant:** Used for planning, code scaffolding, and debugging within the submission window.
- ☐ **MLflow:** Experiment tracking during model tuning.
- ☐ **Aikido:** Security scanning.
- ☐ **Git/GitHub:** Version control and collaboration.
- ☐ **Streamlit Community Cloud:** Deployment.

10. Learnings & Takeaways

Technically, the project reinforced several lessons. Careful exploratory analysis mattered: a naive reading of the data suggested that unsterilised cats get adopted faster, but this was a confound with age (unsterilised cats are mostly kittens); stratifying by age revealed the true picture and shaped the recommendation logic so the tool never gives welfare-harmful advice. Rigorous model validation mattered too: hyperparameter tuning turned out to be within cross-validation noise, so the simplest strong configuration was chosen and reported honestly (~0.68 AUC), rather than chasing a misleadingly high number. Finally, disciplined engineering (a single shared preprocessing path, a saved feature schema, and unit tests) prevented the train/inference skew bugs that are easy to introduce in ML apps.

11. Acknowledgments

- ☐ PetFinder.my and the Kaggle competition organisers for the dataset.
- ☐ The HackTheKitty organisers.
- ☐ Open-source libraries: XGBoost, SHAP, scikit-learn, Streamlit, Plotly, MLflow.

Submission Checklist

- ☐ Video demo (HD or at least 720p)
- ☐ README.md (prerequisites, run instructions, configuration)

- ☐ Project report (this document, in documentation/)
- ☐ Source code in src/
- ☐ No unrelated files, executables, auto-generated code, or package folders (e.g. node_modules/, build/, dist/, vendor/)